

DDD_system_使用统计功能(公网改用Ali云FC)说明

适用场景：Cloudflare Workers 在中国大陆无法访问，迁移至阿里云函数计算 FC + GitHub 存储的完整指南。费用：¥0（阿里云 FC 每月100万次免费调用，GitHub 私有仓库免费）

架构说明

```
1  用户浏览器
2      ↓ HTTP 请求
3  阿里云函数计算 FC（新加坡节点）
4      ↓ 读写 data.json
5  GitHub 私有仓库（数据库）
```

- **阿里云 FC**：运行统计服务逻辑，提供 `/track`、`/api/stats`、`/report` 三个接口
- **GitHub 私有仓库**：存储一个 `data.json` 文件，充当数据库

第一步：创建 GitHub 私有仓库

1. 打开 github.com，登录账号
2. 右上角 "+" → **New repository**
3. 填写：
 - Repository name: `DDD_UseAliCount`
 - 选择 **Public**（同时在这里发布pages）
 - 勾选 **Add a README file**
4. 点击 **Create repository**

第二步：获取 GitHub Token

1. 右上角头像 → **Settings**
2. 左侧最底部 → **Developer settings**
3. **Personal access tokens** → **Tokens (classic)**
4. Generate new token (classic)
，填写：
 - Note: `ddd-stats`
 - Expiration: **No expiration**
 - 权限：勾选 `repo`（第一个大选项，全勾）
5. 点击 **Generate token**，复制保存 `ghp_` 开头的 token（只显示一次）

第三步：创建阿里云函数

1. 打开 fcnext.console.aliyun.com，注册登录并完成实名认证

2. 点击**创建函数**，选择 **Web 函数**
3. 填写：
 - 函数名称：`ddd-stats-worker`
 - 地域：**新加坡 ap-southeast-1**（无ICP备案必须选境外节点）
 - 运行环境：**Node.js 18**
4. 点击**创建 Web 函数**

第四步：修改触发器认证方式

函数创建后，默认认证方式为「签名认证」，需改为无需认证：

1. 函数页面 → 底部 **触发器** 标签页
2. 找到 `defaultTrigger` → 点右侧**编辑**
3. 将「认证方式」改为 **无需认证**
4. 保存

第五步：配置环境变量

函数页面 → 顶部**编辑环境变量** → 添加以下 3 个变量：

变量名	填入的值
<code>GH_OWNER</code>	你的 GitHub 用户名， 注意如果仓库创建在组织下，应该写组织的名字，如我这里的 dlyyyy-dop，而不是我的账号 adamhtmei
<code>GH_REPO</code>	<code>DDD_UseAliCount</code>
<code>GH_TOKEN</code>	第二步复制的 <code>ghp_xxx</code>

保存。

第六步：部署代码

进入函数 → **代码** 标签页，点右上角**配置 WebIDE** 进入编辑器。

index.js（全部替换）

```
1  const https = require('https');
2  const express = require('express');
3  const app = express();
4  app.use(express.json());
5
6  const EVENT_LABELS = {
7    page_view : '网页打开',
8    pwa_install: 'PWA安装',
9    calc_click : '计算DDDs及使用强度',
10 };
11
12 const CORS = {
13   'Access-Control-Allow-Origin' : '*',
14   'Access-Control-Allow-Methods': 'GET, POST, OPTIONS',
```

```

15   'Access-Control-Allow-Headers': 'Content-Type',
16   };
17
18   const GH_OWNER = process.env.GH_OWNER;
19   const GH_REPO = process.env.GH_REPO;
20   const GH_TOKEN = process.env.GH_TOKEN;
21   const GH_FILE = 'data.json';
22
23   function ghRequest(method, path, body) {
24     return new Promise((resolve, reject) => {
25       const data = body ? JSON.stringify(body) : null;
26       const req = https.request({
27         hostname: 'api.github.com',
28         path,
29         method,
30         headers: {
31           'Authorization': `token ${GH_TOKEN}`,
32           'User-Agent' : 'ddd-stats-fc',
33           'Content-Type' : 'application/json',
34           ...(data ? { 'Content-Length': Buffer.byteLength(data) } : {}),
35         },
36       }, res => {
37         let raw = '';
38         res.on('data', c => raw += c);
39         res.on('end', () => {
40           try { resolve({ status: res.statusCode, body: JSON.parse(raw) }); }
41           catch { resolve({ status: res.statusCode, body: raw }); }
42         });
43       });
44       req.on('error', reject);
45       if (data) req.write(data);
46       req.end();
47     });
48   }
49
50   async function loadData() {
51     const res = await ghRequest('GET',
52       `/repos/${GH_OWNER}/${GH_REPO}/contents/${GH_FILE}`);
53     if (res.status === 404) {
54       return { counts: { page_view: 0, pwa_install: 0, calc_click: 0 }, logs:
55         [], sha: null };
56     }
57     const content = Buffer.from(res.body.content, 'base64').toString('utf-
58       8');
59     return { ...JSON.parse(content), sha: res.body.sha };
60   }
61
62   async function saveData(data) {
63     const { sha, ...payload } = data;
64     const content = Buffer.from(JSON.stringify(payload)).toString('base64');
65     await ghRequest('PUT',
66       `/repos/${GH_OWNER}/${GH_REPO}/contents/${GH_FILE}`, {
67         message: 'update stats',
68         content,
69         ...(sha ? { sha } : {}),
70       });
71   }

```

```

69 app.options('*', (req, res) => {
70   Object.entries(CORS).forEach(([k, v]) => res.setHeader(k, v));
71   res.sendStatus(204);
72 });
73
74 app.post('/track', async (req, res) => {
75   Object.entries(CORS).forEach(([k, v]) => res.setHeader(k, v));
76   const event = (req.body && req.body.event) || '';
77   if (!EVENT_LABELS[event]) {
78     return res.status(400).json({ error: 'unknown event', received: event
79   });
80   }
81   const data = await loadData();
82   data.counts[event] = (data.counts[event] || 0) + 1;
83   data.logs.push({
84     event,
85     label: EVENT_LABELS[event],
86     time : new Date().toISOString().replace('T', ' ').substring(0, 19),
87     ua   : ((req.body && req.body.ua) || '').substring(0, 120),
88   });
89   if (data.logs.length > 2000) data.logs = data.logs.slice(-2000);
90   await saveData(data);
91   res.json({ ok: true, count: data.counts[event] });
92 });
93
94 app.get('/api/stats', async (req, res) => {
95   Object.entries(CORS).forEach(([k, v]) => res.setHeader(k, v));
96   const data = await loadData();
97   const { sha, ...clean } = data;
98   res.json(clean);
99 });
100
101 app.get('/report', async (req, res) => {
102   const data = await loadData();
103   const counts = data.counts;
104   const recent = (data.logs || []).slice(-20).reverse();
105
106   const rows = recent.length > 0
107     ? recent.map(l => `
108       <tr>
109         <td>${l.time} || ''</td>
110         <td>${l.label} || EVENT_LABELS[l.event] || l.event</td>
111         <td class="ua">${(l.ua || '').substring(0, 80)}</td>
112       </tr>`).join('')
113     : '<tr><td colspan="3" style="text-align:center;color:#aaa;padding:20px;">暂无数据</td></tr>';
114
115   const html = `<!DOCTYPE html>
116   <html lang="zh-CN">
117   <head>
118     <meta charset="UTF-8">
119     <meta name="viewport" content="width=device-width,initial-scale=1">
120     <title>DDD数智通 · 使用统计</title>
121     <style>
122       body{font-family:"PingFang SC","Microsoft YaHei",sans-serif;background:#f0f5fa;color:#1a2533;margin:0;padding:20px;}
123       h1{font-size:18px;color:#0d2137;margin-bottom:20px;}
124       .cards{display:flex;gap:16px;flex-wrap:wrap;margin-bottom:28px;}

```

```

124   .card{background:#fff;border-radius:12px;padding:20px 28px;box-shadow:0
125   2px 12px rgba(13,33,55,.1);min-width:160px;text-align:center;}
126   .card .num{font-size:42px;font-weight:900;color:#f39c12;line-height:1;}
127   .card .lbl{font-size:13px;color:#7f8c9a;margin-top:6px;}
128   table{width:100%;border-collapse:collapse;background:#fff;border-
129   radius:12px;overflow:hidden;box-shadow:0 2px 12px rgba(13,33,55,.1);}
130   th{background:#0d2137;color:#fff;padding:10px 14px;font-size:13px;text-
131   align:left;}
132   td{padding:9px 14px;font-size:12px;border-bottom:1px solid #eef2f7;word-
133   break:break-all;}
134   tr:last-child td{border:none;}
135   .ua{color:#7f8c9a;font-size:11px;}
136   a{color:#2e86c1;text-decoration:none;}
137   .hint{font-size:12px;color:#7f8c9a;margin-bottom:12px;}
138 </style>
139 </head>
140 <body>
141 <h1> 🐼 DDD数智通 · 使用统计看板</h1>
142 <div class="cards">
143   <div class="card"><div class="num">${counts.page_view||0}</div><div
144   class="lbl">网页打开次数</div></div>
145   <div class="card"><div class="num">${counts.pwa_install||0}</div><div
146   class="lbl">PWA安装次数</div></div>
147   <div class="card"><div class="num">${counts.calc_click||0}</div><div
148   class="lbl">计算DDDs次数</div></div>
149 </div>
150 <p class="hint">最近20条记录 · <a href="/report">刷新</a></p>
151 <table>
152   <thead><tr><th>时间</th><th>事件</th><th>客户端</th></tr></thead>
153   <tbody>${rows}</tbody>
154 </table>
155 </body></html>`;
156
157   res.setHeader('Content-Type', 'text/html; charset=utf-8');
158   res.send(html);
159 }));
160
161 app.use((req, res) => res.send('请访问 /report 查看统计'));
162
163 app.listen(9000, () => console.log('DDD stats server running on port
164 9000'));

```

package.json (全部替换)

```

1  {
2    "name": "ddd-stats-worker",
3    "version": "1.0.0",
4    "private": true,
5    "scripts": {
6      "start": "node ./index.js"
7    },
8    "dependencies": {
9      "express": "~4.16.1"
10   }
11 }

```

在终端执行

打开终端 (Terminal → New Terminal 或 Ctrl + `), 执行:

```
1 | npm install
```

完成后点右上角「部署代码」按钮。

验证部署成功

①在浏览器控制台 (F12 → Console) 粘贴执行:

javascript

```
1 | fetch('https://ddd-stas-worker-wnumbcyor.ap-southeast-1.fcapp.run/track', {  
2 |   method: 'POST',  
3 |   headers: {'Content-Type': 'application/json'},  
4 |   body: JSON.stringify({event: 'page_view', ua: 'test'})  
5 | }).then(r => r.json()).then(console.log)
```

如果返回 `{ok: true, count: 1}`, 再去 GitHub 仓库刷新, 就能看到 `data.json` 了。

②访问以下地址, 看到统计看板页面即部署成功:

```
1 | https://你的FC地址/report
```

第七步: 修改前端 HTML

打开 `DDD_system.html`, 找到所有 Cloudflare Worker 的地址 (`workers.dev` 结尾), 替换为新的 FC 地址。

例如:

```
1 | // 改之前  
2 | const CF_WORKER_API = "https://ddd-stats-api.adamhtmei.workers.dev"; // 填  
   | 写你部署好的 CF 真实地址  
3 |  
4 | // 改之后  
5 | const CF_WORKER_API = "https://ddd-stas-worker-wnumbcyor.ap-southeast-  
   | 1.fcapp.run"
```

常见报错速查

报错信息	原因	解决方法
<code>MissingRequiredHeader</code>	触发器认证方式为「签名认证」	改为「无需认证」（第四步）
<code>CAExited</code> / <code>npm run start</code>	代码格式与运行时不兼容	使用 Express 格式的 <code>index.js</code> （第六步）
<code>ExternalRedirectForbidden</code>	免费域名不允许重定向	末尾路由改为 <code>res.send(...)</code> 而非 <code>res.redirect(...)</code>

接口说明

接口	方法	说明
<code>/track</code>	POST	上报事件, body: <code>{ event, ua }</code>
<code>/api/stats</code>	GET	获取全量统计数据 (JSON)
<code>/report</code>	GET	可视化统计看板 (HTML页面)

支持的事件类型: `page_view`、`pwa_install`、`calc_click`